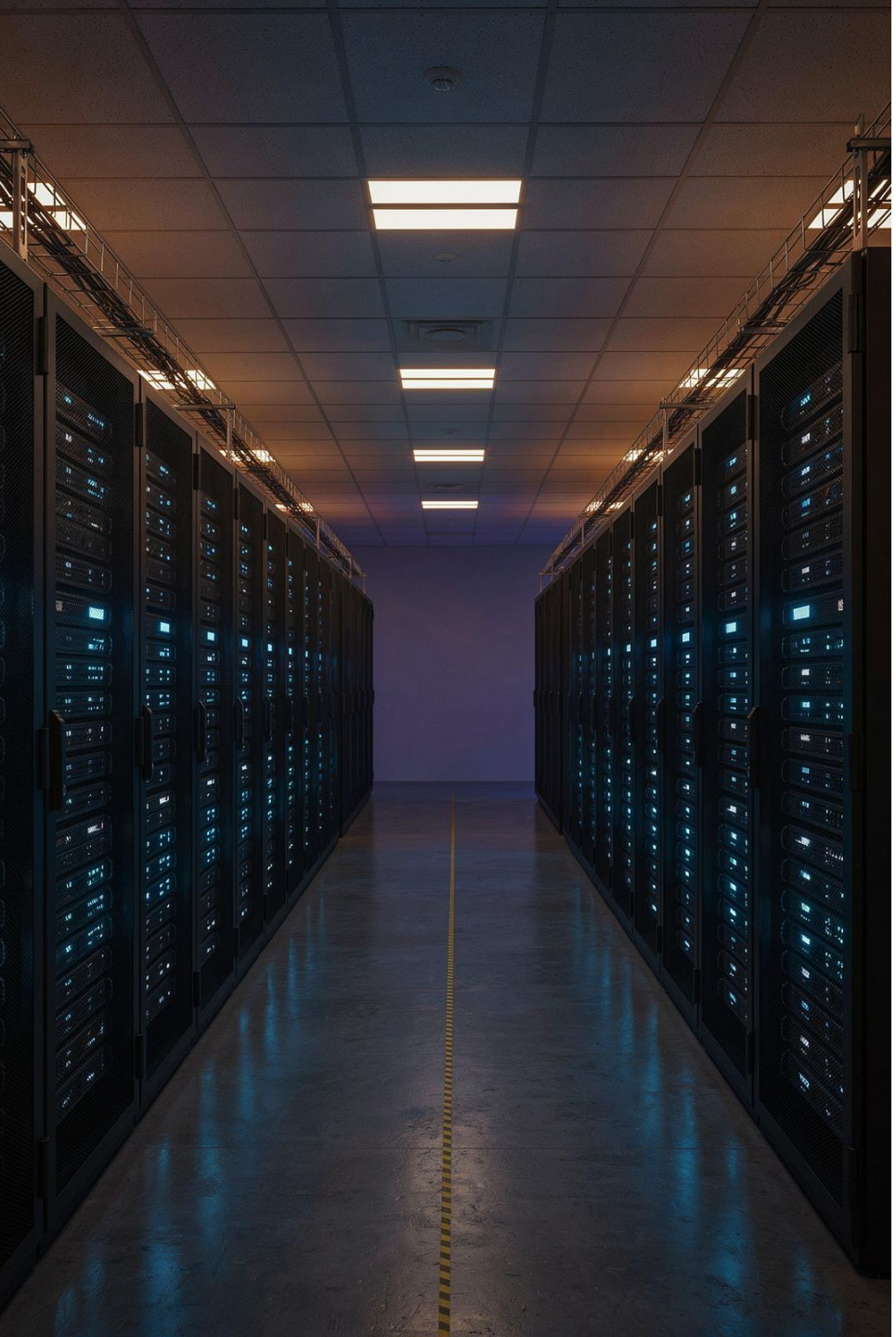


Session 5

Data Foundations, SQL Thinking, and Relational Modeling

Data Foundations, SQL Thinking & Relational Modeling

How enterprise applications store, organize, protect, and query business data using relational databases.

Focus Area

Mortgage servicing platform

Key Outcome

Build a baseline relational model for borrower, loan, payment, servicing, delinquency, and foreclosure data

Why Data Foundations Matter

Every enterprise system depends on trusted data. In a mortgage platform, data answers:

- **Who is the borrower?**
- **Was the latest payment received?**
- **Is the loan delinquent?**
- **Has foreclosure activity started?**

A good application starts with a good data model.

What is Data Modeling?

Converting business requirements into structured data design.

Data modeling helps us decide:

- What data to store
- Which entities are required
- How entities connect
- Rules that protect data quality
- How data will be queried

Mortgage Domain Entities

Key business objects that need to be stored and managed:

Borrower

Loan / Mortgage

Payment

Servicing History

Delinquency

Foreclosure

Entity, Attribute, Relationship

Entity

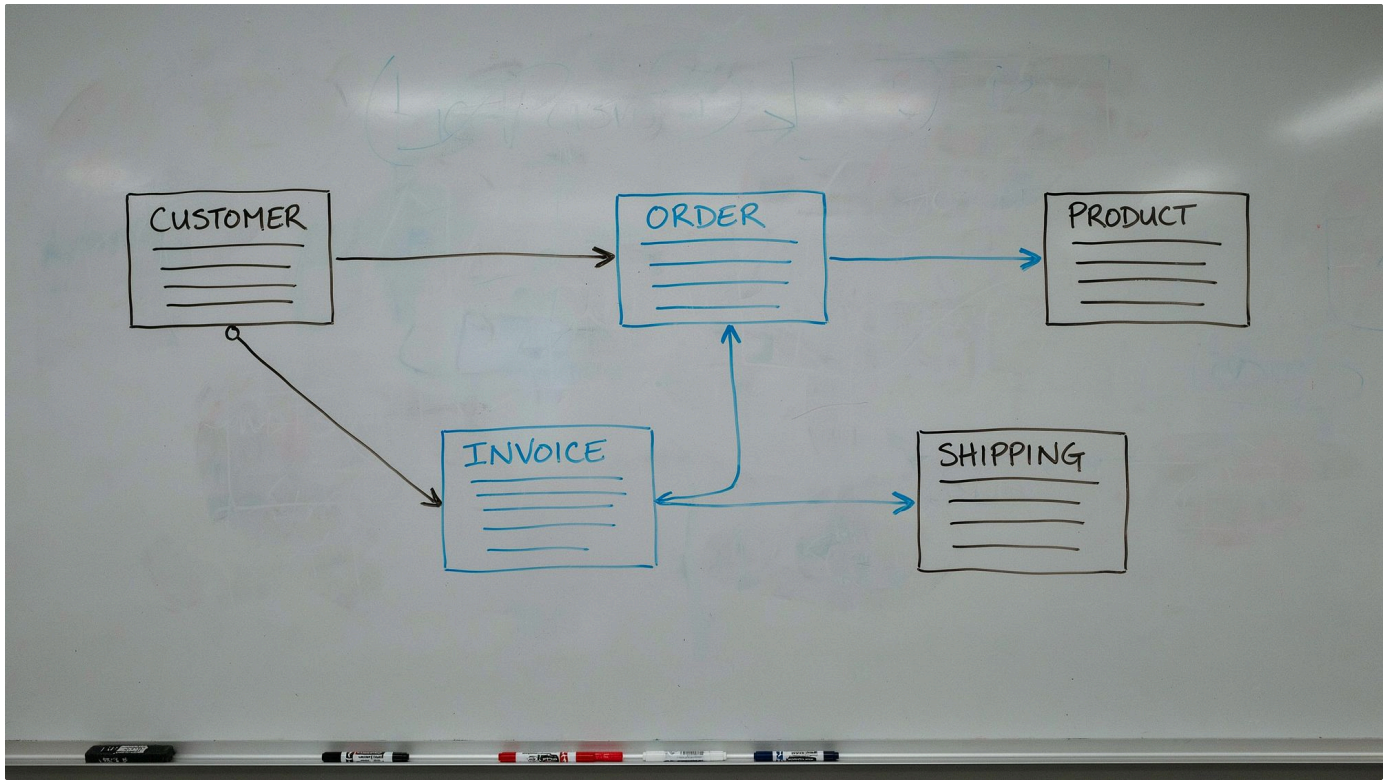
A business object — e.g. **Borrower**

Attributes

borrower_id, first_name, last_name, email, phone

Relationship

One borrower can have one or more loans



Relationship Types

One-to-One

One loan → one loan detail record

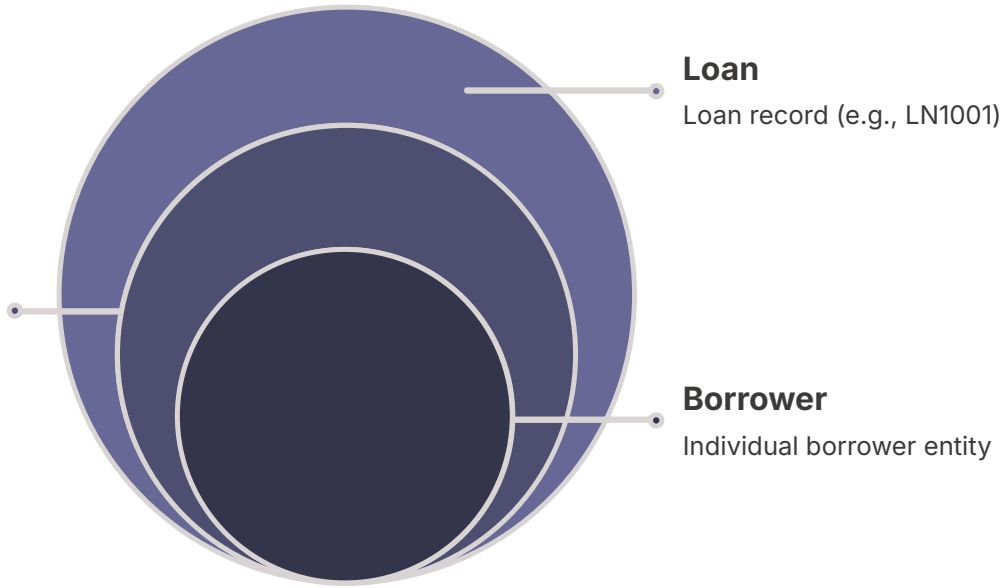
One-to-Many

One loan → many payments

Many-to-Many

One borrower ↔️ multiple loans
Primary borrower + co-borrower on same loan

Borrower-to-Loan Model



Junction Table Pattern

The **Borrower_Loan** table supports primary and co-borrower relationships.

> John → LN1001 → **Primary Borrower**

> Sarah → LN1001 → **Co-Borrower**

What is a Relational Database?

Data stored in structured tables with rows, columns, keys, and relationships.

Borrower Table Borrower details	**Loan Table** Loan details
Payment Table Payment history	

Primary Key

Uniquely identifies one record in a table

borrower_id

loan_id

payment_id

Why not use name? Many borrowers may share the same name.

Unique

No duplicates

Not Null

Always required

Stable

Never changes

Foreign Key

Connects one table to another

Loan table stores **borrower_id** — linking each loan to its borrower.

Foreign keys prevent invalid records — e.g. a loan assigned to a non-existent borrower.

Normalization

Organizing data to reduce duplication and improve consistency.

❌ Bad Design

Loan table contains borrower details *and* all payment details in the same row.

✅ Better Design

Borrower Table

Loan Table

Payment Table

Each table has a clear, single responsibility.

Normal Forms

1	**1NF — Atomic Values** Each payment = a separate row, not a comma-separated list
2	**2NF — Full Key Dependency** Borrower name lives in Borrower table, not repeated in Borrower_Loan
3	**3NF — No Transitive Dependencies** Servicer details in Servicer table; Loan stores only servicer_id

SQL Thinking

Converting a business question into a database query.

Business Question

Show all active loans above $300,000

Entity

Loan

Conditions

status = ACTIVE & amount > 300,000

Columns Needed

loan_number, amount, status

Basic SQL Query

```
SELECT loan_number, original_amount, status
FROM loan
WHERE status = 'ACTIVE'
  AND original_amount > 300000;
```

Retrieves active high-value loans. SQL powers APIs, reports, dashboards, and analytics across enterprise applications.

Joins

Combine related data from multiple tables.

```
SELECT b.first_name, b.last_name, l.loan_number
FROM borrower b
JOIN loan l ON b.borrower_id = l.borrower_id;
```

A JOIN rebuilds business information from normalized tables.

INNER JOIN vs LEFT JOIN

INNER JOIN

Returns only matching records: borrowers with a loan match.

LEFT JOIN

Returns all left table records, including borrowers without loans.

Why it matters

LEFT JOIN is critical for reporting and data validation — ensuring no records are silently excluded.

Aggregation Queries

Summarize data across many records.

```
SELECT loan_id, SUM(amount) AS total_paid
FROM payment
GROUP BY loan_id;
```

COUNT

SUM

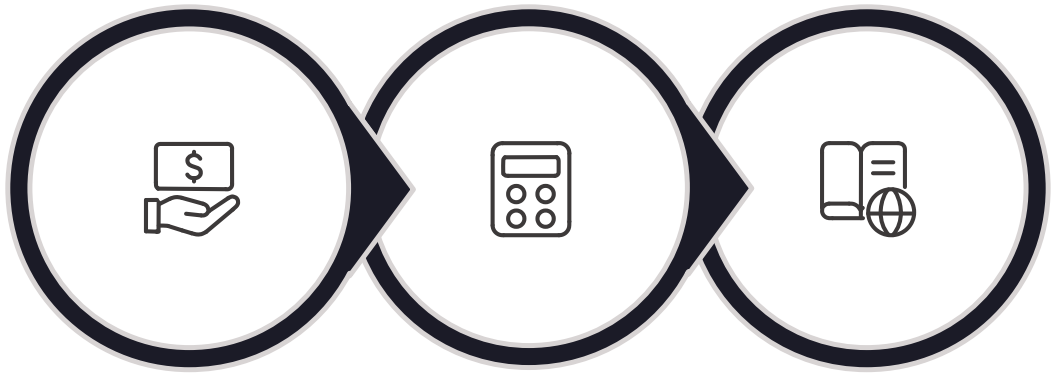
AVG

MIN

MAX

Transactions

Groups multiple operations into one logical unit.



All or Nothing

If one step fails, **all changes roll back** — preventing partial or incorrect financial data.

ACID Properties

Atomicity

All steps succeed or all fail

Consistency

Data rules remain valid

Isolation

Concurrent transactions don't corrupt each other

Durability

Committed data is safely stored

Indexing

Helps the database find data faster

```
CREATE INDEX idx_loan_number
ON loan(loan_number);
```

Best used on columns in:

- WHERE conditions
- JOIN conditions
- ORDER BY clauses

Query Optimization Basics

❌ Avoid

```
SELECT * FROM loan;
```

✅ Prefer

```
SELECT loan_number, status,
  outstanding_balance
FROM loan;
```

Fetch only required columns

Filter early

Use indexes carefully

Avoid unnecessary joins

Check execution plans

EXPLAIN ANALYZE

PostgreSQL tool to understand query performance.

```
EXPLAIN ANALYZE
SELECT * FROM loan
WHERE loan_number = 'LN1001';
```

Sequential Scan

Index Scan

Join Strategy

Sort Operation

Measure first, optimize second.

Enterprise Auditability

Track every data change

- When did loan status change?
- Who changed it?
- What was the previous value?

Standard audit fields:

created_at · created_by · updated_at · updated_by

Critical data may also require **history tables**.

Governance & Retention

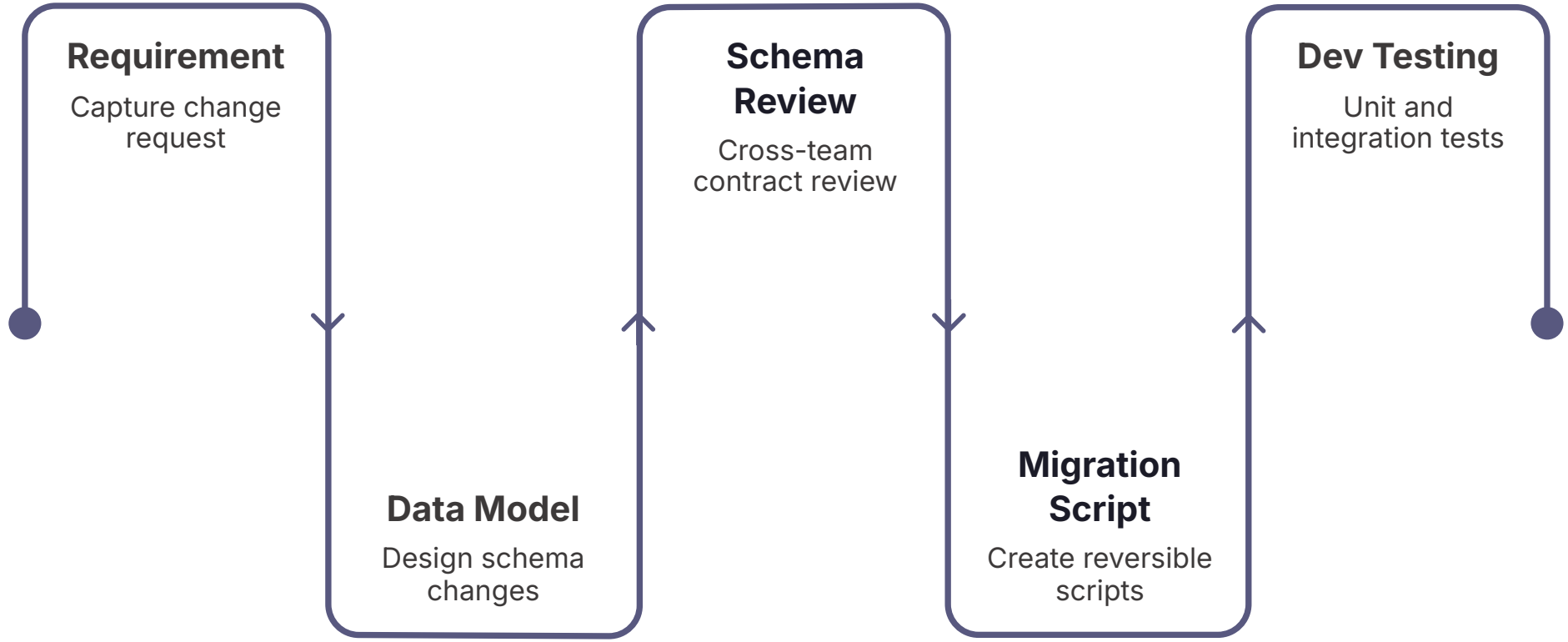
Data governance defines ownership, access, quality, and control.

Mortgage data is sensitive

Retention policies must follow **business, legal, and compliance** requirements.

Schema Review Workflow

Database changes should never go directly to production.



A database schema is a shared contract across applications, reports, and integrations.

DB Migration Workflow

Version-controlled schema changes

01

V001_create_borrower.sql

02

V002_create_loan.sql

03

V003_create_payment.sql

04

V004_add_risk_category.sql

Migration Benefits

- **Repeatable changes**
- **Environment consistency**
- **Audit trail**
- **Rollback planning**
- **Controlled release**

Hands-On Lab

1. Design PostgreSQL schema for mortgage servicing
2. Create borrower-to-loan relationship model
3. Insert sample mortgage data
4. Write SQL queries with joins and aggregates
5. Use indexes to optimize slow mortgage searches

Capstone Progress

Finalized baseline relational model — all core entities in place.

Session Recap

Data Modeling

Entities, attributes, relationships

Keys

Primary keys & foreign keys

Normalization

1NF, 2NF, 3NF

SQL

Queries, joins, aggregations

Transactions & ACID

Data consistency & reliability

Governance

Auditability, retention, migrations

A Strong Enterprise Engineer...

- Understands the business domain
- Designs clean data models
- Protects data integrity
- Optimizes queries & follows governance

The Foundation for Everything Ahead

Today's data model becomes the foundation for:

APIs

Analytics

Security

DevSecOps

Final Capstone

Build it right.
Build it once.

Session 5 Complete — Data Foundations, SQL Thinking, and Relational Modeling